Instituto Tecnológico de Aeronáutica - ITA
CM-204 - Aprendizado Profundo
Aluno: Ulisses Lopes da Silva

Relatório do Laboratório 01 - Descida do Gradiente e Introdução ao PyTorch

---

1 Breve Explicação em Alto Nível da Implementação

1.1 `gradient_descent_numpy`

Para a descida do gradiente em `numpy`, procedeu-se, basicamente, de acordo com o algoritmo dos slides apresentados em aula. Antes, porém, foi importado da biblioteca `math` o parâmetro `inf`, que funcionaria como o "custo inicial".

Em seguida, definiu-se os parâmetros de iterações e de custo, a fim de elaborar o loop, conforme as duas condições de parada (número máximo de iterações ou atingimento do valor *epsilon*). Dentro do loop, procedia-se com a variação de θ_k de acordo com o resultado do gradiente calculado com `gradient_function`. Em seguida, anotava-se o novo valor de θ_k no histórico, acrescentava-se uma unidade no parâmetro de iteração e obtinha-se o novo custo com `cost_function` para a nova comparação do loop.

```
    # Todo: Implement Gradient Descent using pure NumPy
    steps = 0
    cost = inf
    # definindo condicao de parada: enquanto nao alcancar o numero maximo de
       iteracoes e o custo for maior que epsilon
    while steps < max_iterations and cost > epsilon:
        theta = theta - alpha*gradient_function(theta)
        history.append(np.copy(theta))
        steps += 1
        cost = cost_function(theta)
```

1.2 `cost_function_torch`

Para a função `cost_function_torch`, conforme explicitado nos argumentos que ela recebe, era necessário que o parâmetro passado fosse um tensor. Dessa forma, para que o código ficasse eficiente, sem ter que realizar a transformação do vetor de dados t e v a cada chamada de função, foi acrescentado antes do início do teste o seguinte trecho:

```
    # Turning t and v into pytorch tensors
    tensor_t = torch.tensor(t)
    tensor_v = torch.tensor(v)
```

Para a função de custo em Pytorch, uma vez transformados em tensores os parâmetros que a função receberia, procedeu-se conforme o trecho de código abaixo, em que se define o valor de m para ter o número total de amostras. Além disso, a função de custo foi escrita em função dessas variáveis e retornava o valor desse custo, a saber:

$$J(\theta) = \frac{1}{2m} \sum_{k=1}^{m} ||pred - target||^2 \tag{1}$$

onde

$$pred - target = \theta_0 + \theta_1 \cdot \vec{t} - \vec{v} \tag{2}$$

A seguir, o código final:

```
def cost_function_torch(theta: torch.tensor) -> torch.tensor:
# [...]
# Implement the cost function for linear regression using PyTorch
   operations

# Setting variables
m = len(tensor_t)
pred = theta[0] + theta[1]*tensor_t
target = tensor_v

# Calculating the cost in that iteration
cost = (1/(2*m))*(pred - target).pow(2).sum()

return cost
```

1.3 `gradient_descent_torch`

Para a descida do gradiente com Pytorch, procedeu-se, basicamente, conforme o modelo apresentado no código-base. O loop utilizado foi essencialmente igual ao `numpy`, acrescentando-se as operações próprias do Pytorch para o cálculo dos gradientes:

- zerava-se os gradientes;
- calculava-se o vetor de custos e armazenava-se o custo total daquela iteração para comparação no reinício;
- obtinha-se o gradiente;
- executava-se o passo de otimização; e
- anotava-se no histórico os novos valores de $\vec{\theta}$ e somava-se uma unidade ao contador.

```python
def gradient_descent_torch(cost_function: Callable, theta0: torch.tensor,
    alpha: float, epsilon: float,
                        max_iterations: int) -> Tuple[torch.tensor, list]:
# [...]
theta = theta0
history = [theta0.detach().clone()]
optimizer = torch.optim.SGD([theta], lr=alpha) # Stochastic Gradient
    Descent optimizer

# Todo: Implement Gradient Descent using PyTorch
from math import inf

# Setting initial params
steps = 0
loss = inf

# initializing the loop
while steps < max_iterations and loss > epsilon:

    # clearing gradients
    optimizer.zero_grad()

    # computing the loss in that iteration
    # "cost_function" is a param, which receives cost_function_torch as
        argument
    current_loss = cost_function(theta)

    # Updating the loss param
    loss = current_loss.item()

    # computing the gradient
    current_loss.backward()

    # optimization step
    optimizer.step()

    # saving on history
    history.append(theta.detach().clone())


    # updating params
    steps += 1


return theta, history
```

2 Figuras Comprovando Funcionamento do Código

2.1 `gradient_descent_numpy`

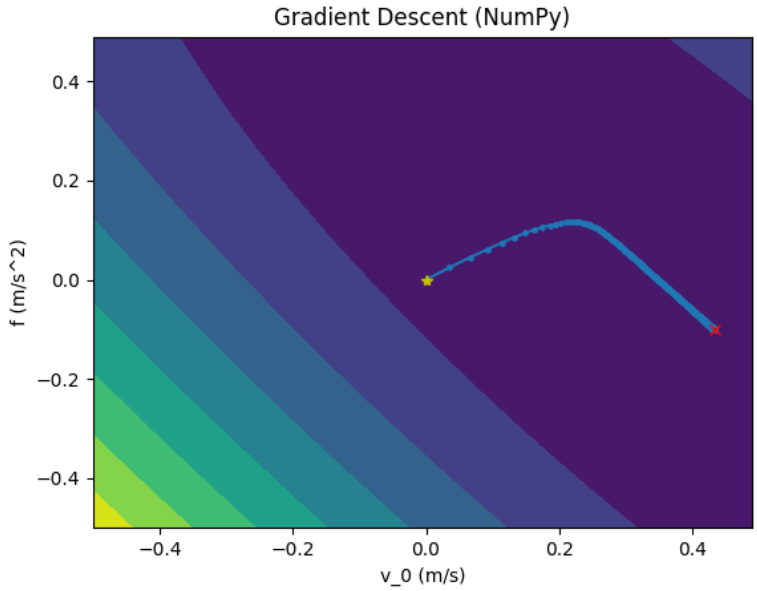

Fig. 1: Descida do gradiente obtida com Numpy. Gradient Descent solution: [0.43337067 -0.10101846]

2.2 `gradient_descent_torch`

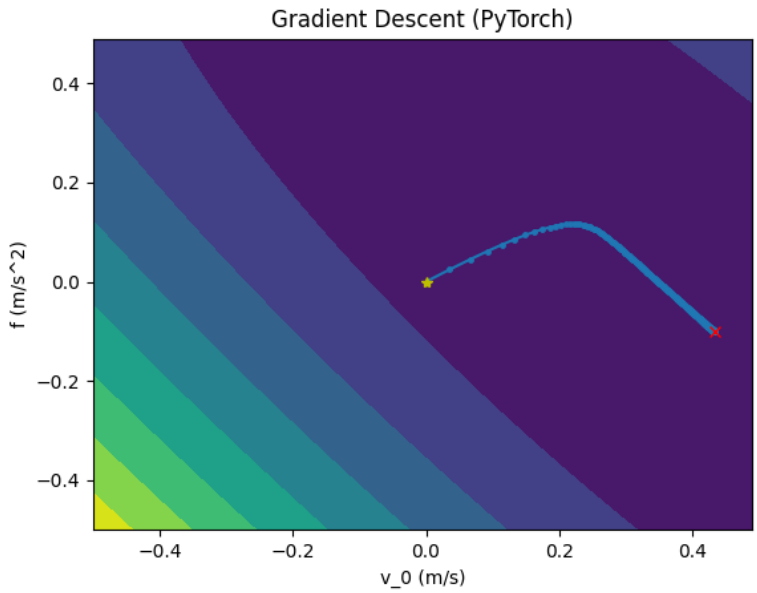

Fig. 2: Descida do gradiente obtida com Pytorch. Gradient Descent solution: tensor([0.4334, -0.1010])

2.3 Comparação dos métodos

- Comparação de otimização

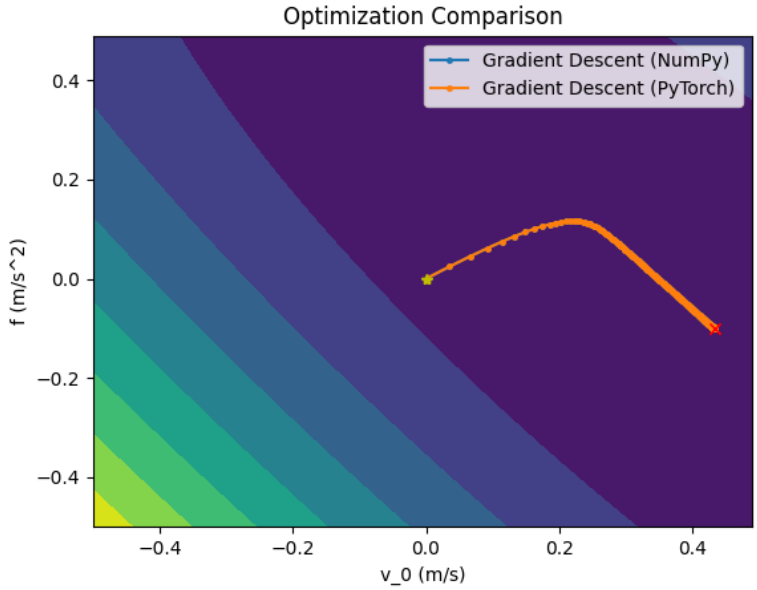

Fig. 3: Comparação de ambos os métodos. Note que os gráficos se sobrepõem perfeitamente.

- Curva de fitting

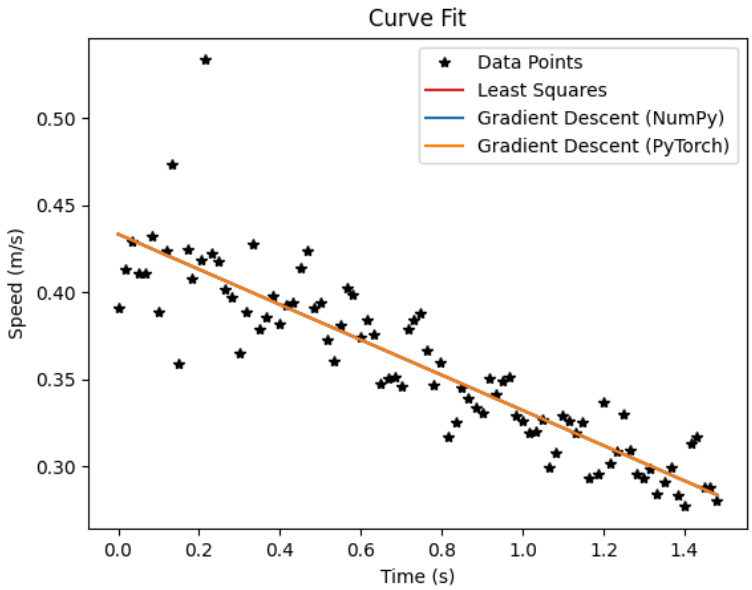

Fig. 4: Curva de ajuste resultate da curva aos pontos

- Quadro comparativo

Tabela 1: Linear regression parameters obtained by the optimization methods

Method	**v0**	**f**
Least-Squares Method	0.433373	-0.101021
Gradient Descent (NumPy)	0.433371	-0.101018
Gradient Descent (PyTorch)	0.433371	-0.101018

3 Discussão dos Resultados

Como se observa nas imagens anteriores, tanto o método de descida do gradiente implementado em NumPy quanto o método utilizando PyTorch apresentaram resultados muito próximos aos obtidos pelo método analítico dos mínimos quadrados, com concordância de até quatro casas decimais. Esse resultado evidencia a eficácia da descida do gradiente como método numérico de otimização, especialmente em problemas nos quais não há solução analítica simples ou viável. No contexto deste laboratório, como o ajuste realizado corresponde a uma regressão linear simples, o método analítico dos mínimos quadrados mostrou-se adequado e eficiente. Entretanto, em problemas mais gerais e complexos, como aqueles frequentemente encontrados em *Deep Learning*, a descida do gradiente torna-se uma ferramenta fundamental, justamente por sua capacidade de generalização e aplicação a modelos cuja solução analítica é inexistente ou impraticável.